n8n Schulung: Fortgeschrittene Workflow-Automatisierung

Inhaltsübersicht

1. Architektur & Deployment-Strategien
2. Erweiterte Node-Konzepte
3. Fehlerbehandlung & Resilienz
4. Custom Code: JavaScript & Python in n8n
5. AI-Integration & Agenten-Workflows
6. Sicherheit & Secrets Management
7. Performance-Optimierung
8. Versionskontrolle & Team-Workflows
9. Praxisprojekte & Übungen

1. Architektur & Deployment-Strategien

1.1 Self-Hosted vs. n8n Cloud

Kriterium	Self-Hosted	n8n Cloud
Datenkontrolle	✅ Vollständig	⚠️ EU/US-Region wählbar
Setup-Aufwand	Hoch	Minimal
Skalierung	Manuell (Docker/K8s)	Automatisch
Custom Nodes	✅ Ja	❌ Nein
Kosten	Infrastruktur	Execution-basiert

1.2 Docker-Deployment (Empfohlen für Produktion)

```yaml
# docker-compose.yml — Produktions-Setup
version: '3.8'
services:
  n8n:
    image: n8nio/n8n
```

2. Erweiterte Node-Konzepte

2.1 Der HTTP Request Node – Fortgeschrittene Nutzung

```
// Beispiel: Pagination mit Loop
{
  "node": "HTTP Request",
  "settings": {
    "url": "https://api.example.com/items",
    "queryParameters": {
      "page": "={{ $json.currentPage }}",
      "limit": "100"
    },
    "authentication": "headerAuth",
    "header": "Authorization: Bearer {{ $credentials.token }}"
  }
}
```

Pagination-Pattern mit SplitInBatches:

3. Fehlerbehandlung & Resilienz

3.1 Error Trigger Workflow

Erstelle einen dedizierten **Error Handler Workflow**:

```
[Error Trigger]
  → [Set Node: Fehler-Details aufbereiten]
     {
       "workflow": "={{ $json.workflow.name }}",
       "node": "={{ $json.execution.lastNodeExecuted }}",
       "error": "={{ $json.execution.error.message }}",
       "timestamp": "={{ $now.toISO() }}",
       "executionUrl": "=https://n8n.company.com/execution/{{ $json.execution.id }}"
     }
  → [Slack / E-Mail: Alert senden]
  → [Optionally: Retry-Logik]
```

Aktivierung im Workflow-Setting:

```
Settings → Error Workflow → [Dein Error Handler auswählen]
```

4. Custom Code: JavaScript & Python in n8n

4.1 Code Node (JavaScript) – Fortgeschrittene Patterns

```javascript
// Alle eingehenden Items verarbeiten
const results = [];

for (const item of $input.all()) {
  const data = item.json;

  // Komplexe Transformation
  const transformed = {
    id: data.id,
    fullName: `${data.firstName} ${data.lastName}`.trim(),
    emailDomain: data.email?.split('@')[1] ?? 'unknown',
    tags: data.tags?.filter(t => t.active).map(t => t.name) ?? [],
    processedAt: new Date().toISOString()
  };

  results.push({ json: transformed });
}
```

5. AI-Integration & Agenten-Workflows

5.1 AI-Node Architektur in n8n

n8n nutzt **LangChain** unter der Haube für alle KI-Nodes:

```
[Trigger / Chat Message]
  → [AI Agent Node]
        ├── Chat Model: OpenAI GPT-4 / Claude / Gemini
        ├── Memory: Window Buffer / Postgres / Redis
        └── Tools:
              ├── [HTTP Request Tool]
              ├── [Code Tool]
              ├── [n8n Workflow Tool]
              └── [Calculator / Web Search]
  → [Response]
```

5.2 Einfacher KI-Agent

```
// AI Agent Konfiguration
{
  "agent": "conversationalAgent"
```

6. Sicherheit & Secrets Management

6.1 Credentials Best Practices

```
✅ DO:
- Alle API-Keys als n8n Credentials speichern
- Credentials nie direkt in Expressions hartkodieren
- Separaten n8n-User pro Integrations-Team anlegen
- Credentials regelmäßig rotieren

❌ DON'T:
- API-Keys als Workflow-Variablen speichern
- Credentials in Workflow-Namen oder Beschreibungen schreiben
- Admin-Account für automatisierte Workflows nutzen
```

6.2 External Secrets Store

```
# AWS Secrets Manager Integration
EXTERNAL_SECRETS_PROVIDER=aws
AWS_REGION=eu-central-1
```

7. Performance-Optimierung

7.1 Batch-Verarbeitung

```
// SplitInBatches Node für große Datensätze
[1000 Items]
  → [SplitInBatches: batchSize=50]
  → [Verarbeitung (50 Items parallel)]
  → [Loop zurück bis alle verarbeitet]
  → [Merge: alle Ergebnisse]
```

Konfiguration:

```
{
  "batchSize": 50,
  "options": {
    "reset": false
  }
}
```

8. Versionskontrolle & Team-Workflows

8.1 Git-Integration einrichten

```
# In n8n Settings → Version Control

# Git-Repository verbinden:
REPOSITORY_URL=git@github.com:company/n8n-workflows.git
GIT_BRANCH=main
```

Workflow:

```
Änderungen in n8n vornehmen
  → "Push" in n8n UI
  → Commit im Git-Repo
  → Code Review via Pull Request
  → Merge → Deployment auf Prod-Instanz via "Pull"
```

8.2 Environments verwalten

9. Praxisprojekte & Übungen

Projekt 1: Intelligentes Ticket-Routing System ⭐⭐

Ziel: Eingehende Support-Tickets automatisch kategorisieren und routen.

```
[Webhook: Neues Ticket]
  → [AI Node: Kategorie & Priorität ermitteln]
     Prompt: "Analysiere dieses Support-Ticket und bestimme:
              - Kategorie: bug/feature/billing/general
              - Priorität: low/medium/high/critical
              Antworte nur als JSON."
  → [Switch Node: nach Kategorie]
      ├── bug      → [Jira: Issue erstellen]
      ├── billing  → [Slack: #billing-team benachrichtigen]
      └── feature  → [Notion: Feature-Backlog eintragen]
  → [E-Mail: Bestätigung an Kunden]
```

Übungsaufgaben:

1. Füge einen Human-in-the-Loop-Schritt für "critical" Tickets hinzu

Projekt 2: Automatisierte Daten-Pipeline ⭐⭐⭐

Ziel: Täglich Daten aus mehreren Quellen aggregieren, transformieren und in eine Datenbank laden.

```
[Cron: täglich 02:00 Uhr]
  → [Parallel: Daten aus 3 APIs holen]
        ├── [API 1: Salesforce Deals]
        ├── [API 2: Google Analytics]
        └── [API 3: Stripe Revenue]
  → [Merge: alle Daten zusammenführen]
  → [Code Node: Datentransformation & Aggregation]
  → [PostgreSQL: Daten upserten]
  → [IF: Anomalie erkannt?]
        ├── Nein → [Slack: Erfolgs-Notification]
        └── Ja  → [Slack: Alert + Detail-Report per E-Mail]
```

Übungsaufgaben:

1. Füge Datenvalidierung mit aussagekräftigen Fehlermeldungen hinzu

Projekt 3: KI-gestützter Content-Workflow ⭐⭐⭐

Ziel: Aus einem Keyword automatisch Blog-Post erstellen, optimieren und veröffentlichen.

```
[Formular: Keyword eingeben]
  → [AI: Outline generieren]
  → [Warte auf Freigabe (Human-in-the-Loop)]
  → [AI: Vollständigen Artikel schreiben]
  → [AI: SEO-Analyse & Optimierung]
  → [Bild: DALL-E / Midjourney via API]
  → [WordPress / CMS: Als Entwurf speichern]
  → [Slack: "Bereit zur Überprüfung" + Link]
```

10. Checklisten & Referenz

✅ Produktions-Readiness Checkliste

Workflow-Qualität:

- [ ] Sinnvoller Workflow-Name (kebab-case, z.B. `customer-invoice-generator`)
- [ ] Beschreibung mit Owner, Trigger, Output vorhanden
- [ ] Tags für Kategorisierung gesetzt
- [ ] Alle Nodes beschriftet

Fehlerbehandlung:

- [ ] Error Handler Workflow zugewiesen
- [ ] Kritische Nodes: "Continue on Error" konfiguriert
- [ ] Fallback-Logik für externe API-Ausfälle implementiert
- [ ] Alerting bei Workflow-Fehlern eingerichtet

📚 Weiterführende Ressourcen

Ressource	URL	Beschreibung
Offizielle Docs	docs.n8n.io	Vollständige Dokumentation
Community Forum	community.n8n.io	Fragen & Antworten
Workflow-Templates	n8n.io/workflows	7.000+ fertige Workflows
YouTube-Kanal	youtube.com/@n8n-io	Video-Tutorials
GitHub	github.com/n8n-io/n8n	Source Code & Issues

🔑 Wichtige Expressions Quick-Reference

```
// Input-Daten
$json                           // Aktuelles Item
$input.all()                    // Alle Items
$input.first()                  // Erstes Item
$input.last()                   // Letztes Item

// Auf andere Nodes zugreifen
$node["NodeName"].json          // Daten eines spezifischen Nodes
$items("NodeName")              // Alle Items eines Nodes

// Workflow-Kontext
$workflow.name                  // Workflow-Name
$execution.id                   // Aktuelle Execution-ID
$now                            // Aktueller Zeitpunkt (Luxon-Objekt)

// Datum & Zeit
$now.toISO()                    // ISO-String: "2025-03-05T10:30:00.000Z"
$now.toFormat('dd.MM.yyyy')     // Deutsch: "05.03.2025"
$now.minus({ days: 7 })         // Vor 7 Tagen
$now.plus({ hours: 2 })         // In 2 Stunden

// String-Operationen
$json.text?.toLowerCase()       // Optional chaining
$json.name?.split(' ')[0]       // Vorname extrahieren
`Hallo ${$json.name}!`          // Template-String

// Zahlen
Math.round($json.price * 1.19) // MwSt-Berechnung
Math.max(...$items().map(i => i.json.value)) // Maximum
```

Schulungsinhalt erstellt für n8n v1.x – Stand März 2025

Alle Beispiele sind produktionserprobt und für Fortgeschrittene konzipiert.